

EXECUTIVE OS v4.0

AUTO-PROVISIONING ENGINE

Self-Extending Agent Capabilities — Developer Specification

Capability Gap Detection • Conversational OAuth Provisioning • Server Catalog Discovery

Three-Layer Self-Extension Architecture

Addendum to the Executive OS v4.0 Blueprint • CONFIDENTIAL

Table of Contents

1. Why This Exists — The Self-Extension Problem
2. Three-Layer Architecture Overview
3. Layer 1: Capability Gap Detection
4. Layer 2: Conversational OAuth Auto-Provisioning
5. Layer 3: Remote Server Discovery Catalog
6. Database Schema Additions
7. Pydantic Contracts
8. API Endpoints
9. Brain Plane Modifications
10. Hands Plane: Provisioning Pipeline
11. Gateway Plane: Auth Callback Handler
12. Server Catalog Format & Management
13. Security Model
14. User Experience Flows
15. Error Handling & Edge Cases
16. Sprint Integration — Where This Fits
17. Testing Strategy
- A. Appendix: Full State Machine
- B. Appendix: WhatsApp Message Templates

1. Why This Exists — The Self-Extension Problem

The current Executive OS architecture treats MCP server deployment as a team-driven, pre-planned process. The Deployment Plan specifies 40 servers across 6 waves, each requiring manual server image building, manifest registration, OAuth client creation, and capability probing. The agent cannot acquire new capabilities on its own.

This creates three problems:

- **Capability ceiling:** The agent can only use tools the development team has pre-deployed. If a user needs a tool that isn't in the 40-server fleet, the agent is helpless. It can suggest the user try a different approach, but it can't solve the problem.
- **Slow capability growth:** Adding a new server requires engineering work, a deployment cycle, and a release. The time from 'user needs capability X' to 'agent has capability X' is weeks to months.
- **User friction:** Even for pre-deployed servers, the user must go through an onboarding flow or wait for the agent to suggest a connection. The agent doesn't proactively reason about what tools would help it complete a task.

The Auto-Provisioning Engine solves all three by enabling the agent to: (1) detect when it's missing a capability during task planning, (2) automatically provision pre-deployed but unconnected servers via conversational OAuth, and (3) discover and deploy entirely new servers from a curated remote catalog.

DESIGN GOAL: From the user's perspective, the agent should feel like it can learn new skills on demand. The user says 'book me a flight,' the agent says 'let me set that up,' the user taps one link, and 30 seconds later the agent has flight booking capability. No technical knowledge required. No manual configuration. One tap.

2. Three-Layer Architecture Overview

The Auto-Provisioning Engine operates across all three planes of the Executive OS architecture:

Layer	What It Does	Where It Lives	When Built
Layer 1: Capability Gap Detection	Brain detects 'I need a tool I don't have'	Brain Plane (React Planning)	Months 7-9
Layer 2: Conversational Auto-Provisioning	Brain sends OAuth link, sends to user via WhatsApp	Hands Plane (react-walk-provisioning)	Months 7-9
Layer 3: Remote Server Discovery	Queries a hosted catalog of vetted MCP servers	Hands Plane (extended React Planning)	Months 13-15

Critical constraint: Layers 1 and 2 are pre-launch (Months 7–9). Layer 3 is post-launch (Months 13–15). Layer 2 is the core of the engine — it's what makes 'one-tap capability acquisition' work. Layer 1 is the intelligence that triggers it. Layer 3 is the long-term growth mechanism.

2.1 Data Flow

Complete flow from user request to new capability online:

```
User: 'Book me a flight to SF next Thursday'
■
■■ Brain Plane: ReAct planning → needs search_flights tool
■■ Brain Plane: Tool Registry lookup → NOT FOUND
■■ Brain Plane: TOOLS.md catalog lookup → FOUND: duffel-mcp (not connected)
■■ Brain Plane: Generates tool_call: provision_server(server_id='duffel-mcp')
■
■■ Hands Plane: ProvisioningPipeline receives provision_server call
■■ Hands Plane: Looks up duffel-mcp in server_catalog → auth_type: api_key
■■ Hands Plane: Determines auth flow → 'ask user for API key' OR 'OAuth redirect'
■■ Hands Plane: Creates provisioning_request record (status: awaiting_auth)
■■ Hands Plane: Returns auth_required response to Brain
■
■■ Brain Plane: Generates user-facing message with auth link/instructions
■■ Gateway Plane: Delivers message to WhatsApp
■
■■ User: Taps OAuth link → authorizes in provider's UI → redirected back
■■ Gateway Plane: OAuth callback received at /api/v1/provision/callback
■■ Gateway Plane: Stores token, notifies ProvisioningPipeline
■
■■ Hands Plane: ProvisioningPipeline activates server
■■ Hands Plane: Starts MCP server (sidecar or microservice)
■■ Hands Plane: Runs capability probe → discovers tools
■■ Hands Plane: Normalizes tools → registers in ToolRegistry
■■ Hands Plane: Updates mcp_user_servers, flags TOOLS.md for regen
■■ Hands Plane: Updates provisioning_request (status: active)
■
■■ Brain Plane: Receives 'server provisioned' event
■■ Brain Plane: Retries original task with new tools available
■■ Brain Plane: search_flights(origin='JFK', dest='SFO', date='2026-02-26')
■
```

■■ User receives flight options – capability acquired and used in one conversation

3. Layer 1: Capability Gap Detection

The Brain Plane must recognize when it needs a tool it doesn't have, and reason about how to acquire it.

3.1 Current Behavior (Without This Engine)

When the ReAct engine plans a task and no suitable tool exists, it generates a text response: 'I don't have the ability to book flights. You might want to try Kayak or Google Flights directly.' This is a dead end — the agent gives up and the user leaves the conversation to solve the problem themselves.

3.2 New Behavior (With This Engine)

The ReAct engine adds a new step to its planning loop:

```
# Modified ReAct planning loop
async def plan_step(task: Task, context: Context) -> Action:
# Step 1: Identify required capabilities
required_tools = identify_required_tools(task)
# Step 2: Check ToolRegistry for available tools
available = tool_registry.list_for_context(context.tier)
missing = [t for t in required_tools if not find_match(t, available)]
# Step 3: NEW – If tools are missing, search the capability catalog
if missing:
    catalog_matches = await search_capability_catalog(
        missing_capabilities=missing,
        user_id=context.user_id
    )
    if catalog_matches:
        # Found a server that can provide the missing capability
        best_match = catalog_matches[0]
        return Action(
            type=ActionType.PROVISION_SERVER,
            server_id=best_match.server_id,
            reason=f'Need {missing[0]} capability for this task',
            original_task=task # Retry after provisioning
        )
# Step 4: If no catalog match, fall back to helpful suggestion
if missing:
    return Action(type=ActionType.INFORM_USER_LIMITATION, ...)
```

3.3 Capability Catalog in TOOLS.md

TOOLS.md currently lists connected servers. Extend it with a catalog section listing available-but-not-connected servers:

```
## Connected MCP Servers
- google-calendar-mcp: ACTIVE | tools: list_events, create_event, ...
- notion-mcp: ACTIVE | tools: search_pages, create_page, ...
## Available Servers (Not Connected)
```

- duffel-mcp: Flight search and booking | auth: api_key | setup: 30 sec
- zoom-mcp: Meeting management, recordings, transcripts | auth: oauth | setup: 30 sec
- plaid-mcp: Bank balances and transactions | auth: plaid_link | setup: 60 sec
- tesla-mcp: Vehicle control, charge status, climate | auth: tesla_sso | setup: 45 sec
- [... full catalog ...]

The Brain Plane reads the 'Available Servers' section during planning. When it encounters a capability gap, it matches required capabilities against this catalog using semantic similarity (not exact string match). The LLM's reasoning handles the matching: 'I need to book a flight. The catalog has duffel-mcp which does flight search and booking. That matches.'

3.4 Capability Matching Rules

Rule	Description	Example
Direct match	User's request directly maps to a catalog server's description	'Book a flight' → duffel-mcp
Inferred match	LLM reasons that a server could help even if not explicitly requested	'Plan my trip' → duffel-mcp + booking-mcp
No match	No catalog server can fulfill the capability gap	'Trade stocks' → no match (no brokerage server in catalog)
Already offered	Server was suggested to user before and they declined	Don't suggest again for 7 days unless user explicitly asks
Plan-gated	Server exists but user's plan doesn't include it	Suggest upgrade: 'This requires the Professional plan.'

4. Layer 2: Conversational OAuth Auto-Provisioning

The core of the engine. Handles the entire lifecycle from 'agent needs a server' to 'server is running and tools are registered' through a single conversational flow.

4.1 Provisioning Pipeline

The ProvisioningPipeline is a new component in the Hands Plane. It's a state machine that manages the lifecycle of a server provisioning request:

State	Description	Transitions To	Timeout
INITIATED	Brain requested provisioning. Pipeline looks up server in catalog.	AWAITING_AUTH, FAILED	10 seconds
AWAITING_AUTH	Auth link/instructions sent to user. Waiting for callback.	AUTH_RECEIVED, EXPIRED, CANCELED	—
AUTH_RECEIVED	OAuth callback received or user provided API key.	PROVISIONING, FAILED	5 seconds
PROVISIONING	Server starting, capability probe running, tools being installed.	ACTIVE, FAILED	60 seconds
ACTIVE	Server is running, tools are in ToolRegistry. Brain can use them.	—	—
FAILED	Error at any stage. Error details logged. User notified.	INITIATED (retry)	—
EXPIRED	User didn't complete auth within timeout. Notified.	INITIATED (retry)	—
CANCELED	User explicitly declined ('No thanks', 'Not now').	INITIATED	—

4.2 Auth Flow Handlers

Different servers require different auth flows. The ProvisioningPipeline delegates to auth-type-specific handlers:

Auth Type	Handler	User Experience	Servers Using This
oauth2	OAuthProvisionHandler	User taps link → provider's authorization page → authenticates → redirect.	Google, Microsoft, Slack, Zoom, Salesforce, Dropbox
oauth2_consolidated	OAuthConsolidatedHandler	Same as oauth2, but one authorization page for multiple servers.	Google, Microsoft, Slack, Zoom, Salesforce, Dropbox
api_key	ApiKeyProvisionHandler	Agent asks user to paste API key in chat. User pastes key.	DuPont, Cerner, Base, Braze, Search, Sentry
plaid_link	PlaidLinkProvisionHandler	User taps link → Plaid Link widget → selects bank → authenticates → redirect. Similar to oauth2.	Plaid bank
tesla_sso	TeslaSSOProvisionHandler	User taps link → Tesla SSO login → authenticates → redirect. Uses PKCE. Region-specific.	Tesla
long_lived_token	TokenProvisionHandler	Agent asks user to provide access token from app's settings page.	Home Assistant
pre_provisioned	PreProvisionedHandler	No user auth needed. Server uses platform's pre-provisioned credentials.	Brave Search (pre-provisioned)

4.3 OAuth Redirect Architecture

When the ProvisioningPipeline generates an OAuth link, it creates a short-lived provisioning session and encodes it in the OAuth state parameter:

```
# Generating the OAuth authorization URL
async def generate_auth_url(request: ProvisioningRequest) -> str:
    server = await get_server_catalog_entry(request.server_id)
```



```

# Create provisioning session (short-lived, stored in Redis)
session = ProvisioningSession(
    id=uuid4(),
    user_id=request.user_id,
    server_id=request.server_id,
    request_id=request.id,
    original_task_id=request.original_task_id,
    created_at=utcnow(),
    expires_at=utcnow() + timedelta(minutes=15),
)
await redis.setex(
    f'provision_session:{session.id}',
    900, # 15 min TTL
    session.json()
)
# Build OAuth URL with provisioning session in state
oauth_config = server.oauth_config
params = {
    'client_id': oauth_config.client_id,
    'redirect_uri': f'{BASE_URL}/api/v1/provision/callback',
    'scope': ' '.join(oauth_config.scopes),
    'state': sign_state({
        'session_id': str(session.id),
        'nonce': generate_nonce()
    }),
    'response_type': 'code',
    'access_type': 'offline', # Get refresh token
}
auth_url = f'{oauth_config.auth_endpoint}?{urlencode(params)}'
# Shorten for WhatsApp (long URLs break in mobile keyboards)
short_url = await shorten_url(auth_url, ttl_minutes=15)
return short_url

```

4.4 OAuth Callback Handler

```

# Gateway Plane: /api/v1/provision/callback
async def provision_callback(code: str, state: str) -> RedirectResponse:
    # 1. Verify and decode state
    state_data = verify_state(state) # Signature check + nonce
    session = await redis.get(f'provision_session:{state_data["session_id"]}')
    if not session: raise HTTPException(410, 'Session expired')
    session = ProvisioningSession.parse_raw(session)
    # 2. Exchange code for tokens
    server = await get_server_catalog_entry(session.server_id)
    tokens = await exchange_oauth_code(
        code=code,
        redirect_uri=f'{BASE_URL}/api/v1/provision/callback',
        oauth_config=server.oauth_config
    )
    # 3. Store tokens in existing oauth_tokens table
    await store_oauth_token(

```

```

user_id=session.user_id,
provider=session.server_id, # e.g., 'zoom-mcp'
access_token=tokens.access_token,
refresh_token=tokens.refresh_token,
expires_at=tokens.expires_at
)
# 4. Trigger server provisioning (async)
await provisioning_pipeline.transition(
request_id=session.request_id,
new_state=ProvisioningState.AUTH_RECEIVED
)
# 5. Redirect user to a 'success' page or deep-link back to WhatsApp
return RedirectResponse(f'{BASE_URL}/provision/success?server={session.server_id}')

```

4.5 Server Activation Sequence

After auth is received, the ProvisioningPipeline activates the server:

```

async def activate_server(request: ProvisioningRequest) -> None:
server = await get_server_catalog_entry(request.server_id)
# 1. Start the MCP server (depends on hosting model)
if server.hosting == HostingModel.SIDECAR:
await start_sidecar(server.container_image, request.user_id)
elif server.hosting == HostingModel.INTERNAL_MICROSERVICE:
# Microservice already running; just register this user
await register_user_with_microservice(server.service_url, request.user_id)
elif server.hosting == HostingModel.EXTERNAL:
# External server; just verify connectivity
await verify_external_server(server.endpoint)
# 2. Run capability probe
capabilities = await mcp_client_hub.probe_server(server.server_id)
# 3. Normalize and register tools
for mcp_tool in capabilities.tools:
tool_spec = normalize_mcp_tool(mcp_tool, server.server_id)
tool_registry.register(tool_spec)
# 4. Create mcp_user_servers record
await create_user_server_connection(
user_id=request.user_id,
server_id=server.server_id,
status='active'
)
# 5. Flag TOOLS.md for regeneration
await flag_tools_md_dirty(request.user_id)
# 6. Notify Brain that provisioning is complete
await event_bus.publish(ServerProvisionedEvent(
user_id=request.user_id,
server_id=server.server_id,
tools_added=[t.name for t in capabilities.tools],
original_task_id=request.original_task_id
))

```

5. Layer 3: Remote Server Discovery Catalog

Post-launch capability. Enables the agent to discover and deploy MCP servers beyond the pre-deployed fleet.

5.1 Catalog Architecture

The Remote Server Catalog is a REST API hosted by the Executive OS platform. It serves as a curated registry of vetted MCP servers. The catalog starts with the 40 pre-deployed servers and grows over time as you add new servers and (eventually) accept third-party submissions.

Catalog is NOT the public internet. The agent does not discover arbitrary MCP servers from GitHub or npm. Every server in the catalog has been: security-reviewed, container-imaged, capability-probed, risk-classified, and cost-estimated. This is a curated App Store model, not open discovery.

5.2 Catalog Entry Schema

```
class ServerCatalogEntry(BaseModel):
    server_id: str # Unique ID: 'duffel-mcp'
    name: str # Display name: 'Duffel'
    description: str # What it does (1-2 sentences)
    capabilities: list[str] # Semantic tags: ['flight_search', 'booking']
    category: str # 'travel', 'finance', 'productivity', etc.
    tools: list[str] # Tool names: ['search_flights', 'create_order']
    auth_type: AuthType # oauth2, api_key, plaid_link, etc.
    oauth_config: OAuthConfig | None # If auth_type is oauth2
    hosting: HostingModel # sidecar, internal_microservice, external
    container_image: str | None # ECR image URI (for sidecar/microservice)
    risk_classification: RiskLevel # Highest risk among tools
    estimated_cost_per_call: float # Average API cost
    setup_time_seconds: int # Expected time from auth to active
    min_plan: Plan # Minimum subscription plan required
    is_pre_deployed: bool # True if already in fleet (Waves 1-6)
    version: str # Catalog entry version
    added_at: datetime # When added to catalog
    status: str # 'available', 'beta', 'deprecated'
```

5.3 Discovery Flow (Post-Launch)

When Layer 1 (capability gap detection) finds no match in the local TOOLS.md catalog:

- Brain Plane calls **search_remote_catalog** tool with the missing capability description.
- Hands Plane queries the Remote Catalog API: GET /api/v1/catalog/search?q=flight+booking
- Catalog returns matching entries, ranked by relevance and popularity.
- Brain presents the best match to the user: 'I found a tool that can book flights — Duffel. It takes about 30 seconds to set up. Want me to connect it?'
- If user approves, the same Layer 2 provisioning pipeline runs.
- If the server requires a container image not already deployed, the Hands Plane pulls it from ECR and starts it. This adds 15–30 seconds to setup time.

SECURITY: Remote catalog entries are signed by the platform. The ProvisioningPipeline verifies the signature before deploying any container image. This prevents catalog tampering. Container images are pulled from the platform's own ECR registry, never from public Docker Hub.

6. Database Schema Additions

Two new tables and one extended table:

6.1 provisioning_requests

Column	Type	Description
id	UUID PK	Request identifier
user_id	UUID FK → users	Who requested the provisioning
server_id	TEXT	MCP server ID from catalog
state	ENUM (initiated, awaiting_auth, auth_received, provisioning, active, failed, expired, canceled)	Current state
auth_type	TEXT	Auth flow used (oauth2, api_key, etc.)
trigger	ENUM (capability_gap, user_request, hardware_capability_suggestion)	What triggered the provisioning
original_task_id	UUID NULL	Task that triggered capability gap (for retry after provisioning)
original_message	TEXT NULL	User's original message (for context when retrying)
error_message	TEXT NULL	If state=failed, the error details
retry_count	INT DEFAULT 0	Number of retry attempts
created_at	TIMESTAMPTZ	Request creation
state_changed_at	TIMESTAMPTZ	Last state transition
completed_at	TIMESTAMPTZ NULL	When state reached terminal (active, failed, expired, canceled)
expires_at	TIMESTAMPTZ	When this request expires (15 min from creation)

6.2 server_catalog

Column	Type	Description
server_id	TEXT PK	Unique server identifier (e.g., 'duffel-mcp')
name	TEXT	Display name
description	TEXT	What the server does (1–2 sentences, LLM-readable)
capabilities	TEXT[]	Semantic capability tags for matching
category	TEXT	Server category (travel, finance, productivity, etc.)
tools_list	TEXT[]	List of tool names the server provides
auth_type	TEXT	Authentication method
oauth_config	JSONB NULL	OAuth client_id, auth_endpoint, token_endpoint, scopes
hosting_model	TEXT	sidecar, internal_microservice, external
container_image	TEXT NULL	ECR image URI
risk_level	TEXT	Highest risk classification among tools
estimated_cost_per_call	NUMERIC(10,6)	Average cost per API call
setup_time_seconds	INT	Expected time to provision
min_plan	TEXT	Minimum subscription plan
is_pre_deployed	BOOLEAN	True if part of Waves 1–6
is_active	BOOLEAN DEFAULT TRUE	Whether available for provisioning
popularity_rank	INT	Popularity ranking (updated weekly from analytics)
version	TEXT	Catalog entry version
created_at	TIMESTAMPTZ	When added
updated_at	TIMESTAMPTZ	Last modified

6.3 provisioning_declined (Track Declined Suggestions)

Column	Type	Description
id	UUID PK	Record identifier
user_id	UUID FK → users	User who declined
server_id	TEXT	Server that was suggested
declined_at	TIMESTAMPTZ	When declined
cooldown_until	TIMESTAMPTZ	Don't suggest again until this timestamp (default: +7 days)

7. Pydantic Contracts

Core Models

```
class ProvisioningRequest(BaseModel):
    id: UUID
    user_id: UUID
    server_id: str
    state: ProvisioningState
    auth_type: AuthType
    trigger: ProvisionTrigger
    original_task_id: Optional[UUID] = None
    original_message: Optional[str] = None
    error_message: Optional[str] = None
    retry_count: int = 0
    expires_at: datetime

class ProvisioningState(str, Enum):
    INITIATED = 'initiated'
    AWAITING_AUTH = 'awaiting_auth'
    AUTH_RECEIVED = 'auth_received'
    PROVISIONING = 'provisioning'
    ACTIVE = 'active'
    FAILED = 'failed'
    EXPIRED = 'expired'
    CANCELED = 'canceled'

class ProvisionTrigger(str, Enum):
    CAPABILITY_GAP = 'capability_gap' # Brain detected missing tool
    USER_REQUEST = 'user_request' # User explicitly asked to connect
    ONBOARDING = 'onboarding' # Part of onboarding flow
    CONTEXTUAL_SUGGESTION = 'contextual_suggestion' # Agent suggested proactively

class AuthType(str, Enum):
    OAUTH2 = 'oauth2'
    OAUTH2_CONSOLIDATED = 'oauth2_consolidated'
    API_KEY = 'api_key'
    PLAID_LINK = 'plaid_link'
    TESLA_SSO = 'tesla_sso'
    LONG_LIVED_TOKEN = 'long_lived_token'
    PRE_PROVISIONED = 'pre_provisioned'
```

Events

```
class ServerProvisionedEvent(BaseModel):
    user_id: UUID
    server_id: str
    tools_added: list[str]
    original_task_id: Optional[UUID] = None
    provisioning_time_ms: int

class ProvisioningFailedEvent(BaseModel):
```

```
user_id: UUID
server_id: str
error: str
state_at_failure: ProvisioningState
retry_count: int
```

Internal Tool Spec

```
# This is the tool the Brain calls to trigger provisioning
PROVISION_SERVER_TOOL = ToolSpec(
    name='provision_server',
    description='Connect a new MCP server to give the agent new capabilities. '
    'Use when a task requires tools that are not currently available '
    'but exist in the Available Servers catalog.',
    input_schema={
        'type': 'object',
        'properties': {
            'server_id': {'type': 'string', 'description': 'Server ID from catalog'},
            'reason': {'type': 'string', 'description': 'Why this server is needed'},
        },
        'required': ['server_id', 'reason']
    },
    risk_level=RiskLevel.LOW, # Provisioning itself is low risk
    is_mcp=False, # This is a NATIVE tool, not an MCP tool
    tags=['system', 'provisioning']
)
```


8. API Endpoints

Method	Path	Description	Auth
GET	/api/v1/provision/callback	OAuth callback handler. Receives authorization code from external service (e.g. Google) and exchanges it for an access token. Redirects user to chat page.	User JWT
GET	/api/v1/provision/success	Success landing page after OAuth. Shows 'Connected' message and a deep-link back to chat.	User JWT
GET	/api/v1/provision/expired	Expired session landing page. Shows 'Session expired' message and a deep-link back to chat.	User JWT
POST	/api/v1/provision/api-key	Receives API key submitted by user via chat (internally routed from chat to Provisioning Engine). Stores key in database.	User JWT
GET	/api/v1/catalog/search	Search server catalog by capability, category, or text query. Returns ranked matches.	User JWT
GET	/api/v1/catalog/servers	List all available servers (for catalog browsing). Filtered by user's plan.	User JWT
GET	/api/v1/catalog/servers/:id	Get detailed catalog entry for a specific server.	User JWT
GET	/api/v1/provision/requests	List user's provisioning request history (for debugging).	User JWT
POST	/api/v1/provision/retry/:id	Retry a failed provisioning request.	User JWT
POST	/api/v1/admin/catalog/sync	Sync remote catalog entries to local database (admin).	Admin JWT
POST	/api/v1/admin/catalog/servers	Add new server to catalog (admin).	Admin JWT
PUT	/api/v1/admin/catalog/servers/:id	Update catalog entry (admin).	Admin JWT

9. Brain Plane Modifications

Minimal changes to the Brain Plane. The Brain is NOT aware of provisioning internals — it just has a new tool it can call.

9.1 New Tool Registration

At startup, the ToolRegistry registers the **provision_server** tool alongside native tools. This is a native tool (`is_mcp=False`) that the Brain can call like any other tool. The Brain doesn't know or care that it triggers a multi-step provisioning pipeline behind the scenes.

9.2 New Tool: `search_remote_catalog` (Layer 3 only)

Post-launch addition. A second native tool that queries the remote catalog:

```
SEARCH_CATALOG_TOOL = ToolSpec(
    name='search_remote_catalog',
    description='Search for MCP servers that could provide a capability '
    'not available in the current server fleet. Use only when '
    'provision_server fails because the server_id is not in the catalog.',
    input_schema={
        'type': 'object',
        'properties': {
            'capability': {'type': 'string', 'description': 'What capability is needed'},
        },
        'required': ['capability']
    },
    risk_level=RiskLevel.NONE,
    is_mcp=False,
    tags=['system', 'discovery']
)
```

9.3 TOOLS.md Catalog Section

The TOOLS.md generation (nightly consolidation job) is extended to include the 'Available Servers' catalog section. This section lists every server in the `server_catalog` table that the user has NOT connected, filtered by the user's plan. Format per entry:

```
- {server_id}: {description} | auth: {auth_type} | setup: ~{setup_time}s
```

The Brain reads this catalog during planning. When it identifies a capability gap, it looks here before calling `search_remote_catalog`. Local catalog matches (pre-deployed servers) are preferred over remote matches because they provision faster (no container pull needed).

9.4 Task Retry After Provisioning

When the ProvisioningPipeline publishes a **ServerProvisionedEvent** with an `original_task_id`, the Brain Plane receives it and re-enters the ReAct loop for the original task. The task's context is preserved (original user message, conversation history), but the available tools now include the newly provisioned server's tools. The Brain should NOT re-prompt the user — it should immediately execute the original task with the new tools and deliver the result.

```
# Brain Plane: handling ServerProvisionedEvent
async def on_server_provisioned(event: ServerProvisionedEvent):
    if event.original_task_id:
        # Retrieve original task and its context
        task = await get_task(event.original_task_id)
        context = await rebuild_context(task, include_new_tools=True)
        # Re-enter ReAct loop with new tools available
        result = await react_engine.execute(task, context)
        # Prepend a one-line confirmation
        confirmation = f'{event.server_id} connected! '
        result.text = confirmation + result.text
        # Deliver to user
        await deliver_response(task.conversation_id, result)
```

10. Hands Plane: Provisioning Pipeline

The *ProvisioningPipeline* is a new component in the Hands Plane alongside the MCP Client Hub and Tool Executor.

10.1 Component Location

```
hands_plane/
■ ■ ■ tool_executor.py # Existing
■ ■ ■ mcp_client_hub.py # Existing
■ ■ ■ provisioning/
■ ■ ■ ■ ■ __init__.py
■ ■ ■ ■ ■ pipeline.py # ProvisioningPipeline state machine
■ ■ ■ ■ ■ handlers/
■ ■ ■ ■ ■ ■ ■ ■ ■ __init__.py
■ ■ ■ ■ ■ ■ ■ ■ ■ base.py # AuthHandler base class
■ ■ ■ ■ ■ ■ ■ ■ ■ oauth2.py # OAuthProvisionHandler
■ ■ ■ ■ ■ ■ ■ ■ ■ oauth2_consolidated.py # Multi-server OAuth
■ ■ ■ ■ ■ ■ ■ ■ ■ api_key.py # ApiKeyProvisionHandler
■ ■ ■ ■ ■ ■ ■ ■ ■ plaid_link.py # PlaidLinkProvisionHandler
■ ■ ■ ■ ■ ■ ■ ■ ■ tesla_sso.py # TeslaSSOProvisionHandler
■ ■ ■ ■ ■ ■ ■ ■ ■ pre_provisioned.py # No-auth handler
■ ■ ■ ■ ■ ■ ■ ■ ■ activator.py # Server activation (start, probe, register)
■ ■ ■ ■ ■ ■ ■ ■ ■ catalog.py # Catalog query interface
■ ■ ■ ■ ■ ■ ■ ■ ■ url_shortener.py # Short URL generation for OAuth links
■ ■ ■ ■ ■ ■ ■ ■ ■ native_tools/
■ ■ ■ ■ ■ ■ ■ ■ ■ provision_server.py # Handler for provision_server tool
■ ■ ■ ■ ■ ■ ■ ■ ■ search_catalog.py # Handler for search_remote_catalog tool
```

10.2 Pipeline State Machine

The *ProvisioningPipeline* is an async state machine. Each state transition is atomic and logged. Failed transitions can be retried.

```
class ProvisioningPipeline:
    VALID_TRANSITIONS = {
        ProvisioningState.INITIATED: [
            ProvisioningState.AWAITING_AUTH,
            ProvisioningState.PROVISIONING, # For pre_provisioned auth type
            ProvisioningState.FAILED,
        ],
        ProvisioningState.AWAITING_AUTH: [
            ProvisioningState.AUTH_RECEIVED,
            ProvisioningState.EXPIRED,
            ProvisioningState.CANCELED,
        ],
        ProvisioningState.AUTH_RECEIVED: [
            ProvisioningState.PROVISIONING,
            ProvisioningState.FAILED,
        ],
        ProvisioningState.PROVISIONING: [
```

```
ProvisioningState.ACTIVE,  
ProvisioningState.FAILED,  
],  
ProvisioningState.FAILED: [  
ProvisioningState.INITIATED, # Retry  
],  
ProvisioningState.EXPIRED: [  
ProvisioningState.INITIATED, # Retry  
],  
}
```

```
async def transition(self, request_id: UUID, new_state: ProvisioningState):  
    request = await self.get_request(request_id)  
    if new_state not in self.VALID_TRANSITIONS.get(request.state, []):  
        raise InvalidTransition(f'{request.state} → {new_state}')  
    request.state = new_state  
    request.state_changed_at = utcnow()  
    await self.save_request(request)  
    await self.execute_state_action(request)
```

11. Gateway Plane: Auth Callback Handler

The Gateway Plane receives two new routes:

- **/api/v1/provision/callback** — OAuth callback. Handles all OAuth-type provisioning flows. Already detailed in Section 4.4.
- **/api/v1/provision/success** — Success landing page. Simple HTML page that says 'Connected! You can close this window and return to your chat.' Includes a WhatsApp deep-link (wa.me/{bot_number}) for easy return.
- **/api/v1/provision/expired** — Expired session page. Says 'This link has expired. Ask your assistant to try again.'

These are the ONLY Gateway Plane changes. The provisioning intelligence is in the Hands Plane. The Brain Plane sends messages via the normal message delivery path.

11.1 Success Page Template

```
<!DOCTYPE html>
<html><body>
<h1>Connected!</h1>
<p>{{ server_name }} is now set up. You can close this window.</p>
<a href="https://wa.me/{{ bot_number }}">
Return to Chat
</a>
</body></html>
```

12. Server Catalog Format & Management

12.1 Initial Catalog Seeding

At launch (Month 12), the `server_catalog` table is seeded with all 40 servers from Waves 1–6. Each entry includes `auth_type`, `oauth_config`, `hosting_model`, and all metadata needed for the ProvisioningPipeline to provision them without any code changes.

The seed migration generates catalog entries directly from the existing `mcp_servers` table + the Deployment Plan and Wave 5–6 documents. This is a one-time data migration, not manual entry.

12.2 Catalog Growth Strategy

Phase	Timeline	Catalog Size	Source
Launch	Month 12	40 servers	Waves 1–6 pre-deployed fleet
Post-launch expansion	Months 13–18	60–80 servers	Platform team builds additional servers based on user demand signals
Developer program (beta)	Months 18–24	100–150 servers	Third-party developers submit MCP servers. Platform team reviews and approves
Open marketplace	Month 24+	500+ servers	Self-serve submission with automated security scanning + human review

12.3 Catalog Sync

For Layer 3 (remote discovery), the local `server_catalog` table is periodically synced with a central catalog API. Sync frequency: daily. The sync pulls new entries, updates existing ones, and marks deprecated ones. This ensures the agent always has an up-to-date view of available capabilities even if the local cache is stale.

13. Security Model

13.1 Threat Model

Threat	Mitigation
Malicious OAuth redirect (CSRF)	State parameter is cryptographically signed (HMAC-SHA256) with a nonce. Callback verifies signature.
Token theft from provisioning session	Sessions stored in Redis with 15-min TTL. Session ID is UUID, not guessable. Token exchange requires session ID.
User tricks agent into provisioning a dangerous server	Remote catalog entries are pre-vetted. The agent can ONLY provision servers in the catalog. No arbitrary server creation.
Catalog poisoning (tampered entries)	Catalog entries are signed. ProvisioningPipeline verifies signature before deploying container image.
Cost explosion via mass provisioning	Rate limit: max 5 provisioning requests per user per hour. Max 3 concurrent requests per user.
Provisioning a server the user's plan doesn't cover	Plan check in ProvisioningPipeline before auth flow starts. Free tier limited to Wave 1 servers.
OAuth token stored for wrong user	Provisioning session binds user_id at creation. Callback verifies session.user_id matches before provisioning.

13.2 Rate Limits

Limit	Value	Scope
Provisioning requests per hour	5	Per user
Concurrent provisioning requests	3	Per user
Auth session TTL	15 minutes	Per request
Declined server re-suggestion cooldown	7 days	Per user + server pair
Remote catalog searches per hour	20	Per user
Failed provisioning retries	3 max	Per request

14. User Experience Flows

14.1 Flow A: Capability Gap → OAuth Server

User says: 'Set up a Zoom with the board for Friday at 2 PM'

Agent: I can set that up – I just need to connect to your Zoom account first.

It takes about 30 seconds. Tap the link below to authorize:

■■ Connect Zoom → [short link]

(You'll be redirected to Zoom's login page, then right back here.)

[User taps link, authorizes in Zoom, redirected to success page]

Agent: Zoom connected! ✓

I've created a Zoom meeting for Friday at 2 PM:

■■ Board Meeting

■■ Friday, Feb 20 at 2:00 PM EST

■■ <https://zoom.us/j/12345678>

Want me to send the invite to the board members?

14.2 Flow B: Capability Gap → API Key Server

User says: 'How much has Acme Corp raised?'

Agent: I can look that up with Crunchbase, but I need an API key.

You can get one free at crunchbase.com/data-api:

1. Sign up or log in
2. Go to API Keys in your dashboard
3. Copy your key and paste it here

(I'll store it securely and never share it.)

User: `sk_crunchbase_abc123xyz789`

Agent: Crunchbase connected! ✓

Acme Corp has raised \$47M across 3 rounds:

- Series B: \$30M (Jan 2026, led by Sequoia)
- Series A: \$12M (Mar 2025, led by a16z)
- Seed: \$5M (Sep 2024)

14.3 Flow C: User Declines

Agent: I can book flights if you connect Duffel. Takes 30 seconds.

■■ Connect Duffel → [link]

Or say 'not now' to skip.

User: Not now

Agent: No problem! I can still help you find flights – I'll search the web and share links you can book directly. Want me to look up flights to SF?

The agent records the decline in `provisioning_declined` with a 7-day cooldown. It falls back to whatever it CAN do (web search for flights) rather than giving up entirely.

14.4 Flow D: Auth Expires

Agent: I need to connect Zoom. Tap the link below:

■■ Connect Zoom → [link]

[15 minutes pass, user didn't tap the link]

Agent: The Zoom connection link expired (they only last 15 minutes).

Want me to generate a new one?

15. Error Handling & Edge Cases

Scenario	Detection	Response
OAuth provider returns error	Non-200 response on token exchange	State → FAILED. Agent: 'The connection didn't work. [Provider] returned an error'
Container image pull fails	Docker pull timeout or auth failure	State → FAILED. Retry once. If fails again, agent: 'I'm having trouble setting this up'
Capability probe returns 0 tools	Empty tools list from MCP host	State → FAILED. This indicates a server misconfiguration. Alert admin. Agent: 'The server isn't working properly'
User sends a message while provisioning	Receive provisioning_request in PROCESSING state	Process the new message normally. If it relates to the same task, remind: 'I'm still working on that'
Server provisioned but original task text is null or task expired	original_task_text is null or task_expired	Reprocess the original task. Confirm connection: 'Zoom is now connected! What can I do for you?'
User provides wrong API key	Server probe fails with authentication error	State → FAILED. Agent: 'That key didn't work — Crunchbase returned an authentication error'
Concurrent provisioning for same server	Duplicate request for same server_id	Return existing request instead of creating new one. If existing is EXPIRED or FAILED, reprocess
Plan doesn't cover requested tool	Tool not in plan check fails	Don't start provisioning. Agent: 'Flight booking is available on the Professional plan'

16. Sprint Integration — Where This Fits

Month	Sprint	Deliverable	Dependencies
Month 7	S13–14	Layer 1: Capability gap detection in ReAct engine. TOOL: Brain catalog (Section 4) + Tool: Provisioning server (Section 8)	Brain catalog (Section 4) + Tool: Provisioning server (Section 8)
Month 8	S11–12	Database: provisioning_requests + server_catalog + provisioning_guidelines (tables 8, same sprint)	MCP client (table 8, same sprint)
Month 8	S13–14	Auth handlers: OAuthProvisionHandler, ApiKeyProvisionHandler, OAuthProvisionHandler, OAuthProvisionHandler	OAuthProvisionHandler, OAuthProvisionHandler, OAuthProvisionHandler, OAuthProvisionHandler
Month 9	S15–16	Integration: Brain calls provision_server → Hands provisioning → Wave Gateways deployed (Month 9)	Wave Gateways deployed (Month 9)
Month 9	S17–18	Additional auth handlers: PlaidLinkHandler, TeslaSSOHandler, OAuthProvisionHandler, OAuthProvisionHandler	PlaidLinkHandler, TeslaSSOHandler, OAuthProvisionHandler, OAuthProvisionHandler
Month 10–12		Catalog updated as each Wave 2–4 server deploys. Conversion 2–4 deployment triggers wired to provisioning	Wave 2–4 deployment triggers wired to provisioning
Month 13–15		Layer 3: Remote catalog API. search_remote_catalog tool. Post-hoc system stability. Extended catalog with Wave 5	Post-hoc system stability. Extended catalog with Wave 5

BUILD ORDER: Layer 1 (Month 7) → Pipeline + basic auth handlers (Month 8) → End-to-end integration (Month 9) → Additional auth handlers + catalog seeding (Month 9–12) → Remote discovery (Month 13–15). Total new engineering: approximately 3–4 weeks spread across Months 7–9, parallel with existing MCP client work.

17. Testing Strategy

17.1 Unit Tests

Component	Tests
ProvisioningPipeline	State machine transitions (valid + invalid). Timeout handling. Concurrent request dedup.
OAuthProvisionHandler	URL generation with correct params. State signing + verification. Token exchange mock.
ApiKeyProvisionHandler	Key validation. Encrypted storage. Probe failure handling.
Server Activator	Sidecar start. Microservice registration. Capability probe normalization. Tool registration.
Catalog Search	Semantic matching. Plan-gated filtering. Declined server cooldown.
Capability Gap Detection	ReAct planning with missing tools. Catalog lookup. Fallback to suggestion.

17.2 Integration Tests

- **Happy path:** Brain detects gap → provisions server → auth completes → server active → original task retried → result delivered. Test with mock OAuth provider.
- **Decline path:** Brain suggests → user declines → cooldown set → fallback response → re-suggestion after cooldown.
- **Expiration path:** Auth link sent → 15 min timeout → state=EXPIRED → agent offers new link.
- **Failure + retry path:** Auth succeeds → probe fails → state=FAILED → retry → succeeds.
- **Concurrent requests:** Two messages in quick succession both trigger provisioning for same server → only one request created.
- **Plan gating:** Free tier user tries to provision a Professional-tier server → upgrade suggestion.

17.3 Golden Scenario Tests

Add these to the golden test suite (Component 4 in Operational Blueprint):

- GT-101: 'Book me a flight to SF' with no Duffel connected → agent suggests connecting Duffel.
- GT-102: 'What's my checking balance?' with no Plaid connected → agent suggests connecting Plaid.
- GT-103: 'Set up a Zoom for Friday' with Zoom in catalog but not connected → agent provisions Zoom.
- GT-104: 'Trade AAPL stock' with no brokerage in catalog → agent gracefully declines, suggests alternatives.
- GT-105: 'Connect Slack' (explicit user request) → agent provisions immediately without capability gap reasoning.

A. Appendix: Full State Machine Diagram



B. Appendix: WhatsApp Message Templates

Pre-approved WhatsApp Business API message templates for provisioning flows:

Template: provision_oauth_request

I can {{capability_description}} if you connect {{server_name}}.

It takes about {{setup_time}} seconds. Tap the link below:

■■ Connect {{server_name}} → {{short_url}}

(You'll authorize on {{provider_name}}'s page, then return here.)

Or say 'not now' to skip.

Template: provision_api_key_request

I can {{capability_description}} with {{server_name}}, but I need an API key.

You can get one at {{docs_url}}:

1. Sign up or log in
2. Go to API Keys in your dashboard
3. Copy your key and paste it here

I'll store it securely and never share it.

Template: provision_success

{{server_name}} connected! ✓

Template: provision_expired

The {{server_name}} connection link expired (they last 15 minutes).

Want me to generate a new one?

Template: provision_failed

Something went wrong connecting {{server_name}}: {{error_summary}}.

Want to try again?

Template: provision_plan_gated

{{capability_description}} requires the {{required_plan}} plan.

You're currently on {{current_plan}}. Would you like to upgrade?

— END OF AUTO-PROVISIONING ENGINE SPECIFICATION —

This document is a standalone addendum to the Executive OS v4.0 Blueprint. It specifies a three-layer self-extension architecture enabling the agent to detect capability gaps, provision MCP servers via conversational OAuth, and discover new capabilities from a curated catalog. Schema additions: 3 tables. Sprint mapping: Months 7–15.